Execution, History, and Reduction: A Minimal Operational Kernel for Computation

Flyxion

August 13, 2026

Abstract

Modern computational theory frequently treats programs as static objects whose meaning is determined by membership relations over sets of states. Yet real systems operate through irreversible event histories rather than through timeless configurations. This paper proposes and develops a minimal operational kernel in which event histories constitute the primary structure of computation, while states and abstractions emerge as derived reductions of those histories.

Three primitive operations govern this kernel: *extension*, which appends events to histories and so realizes execution; *merge*, which reconciles compatible histories through a join-like operation; and *reduction*, which compresses histories into derived representations through controlled information loss. From the interaction of these operations arise deterministic replay, distributed convergence, log-structured computation, and constraint-driven dynamics.

The space of executable histories is generated by these three primitive operations, and all computational processes arise as compositions of these generators. The algebra induced by these operations is shown to correspond to a join-semilattice structure over a partially ordered space of histories, admitting a monotone potential whose descent characterizes execution. Histories are further shown to constitute an order-enriched monoidal category whose dual operational representations—acting on states and observables respectively—need not impose identical admissibility constraints on history factorization, revealing an intrinsic directional structure in the algebra of execution.

The framework thereby replaces set-theoretic membership with replayable construction histories, providing a unified account of execution, composition, and abstraction applicable both to contemporary computational architectures and to physical systems governed by locally propagated constraints.

Contents

1 The Failure of State Ontology **5**

1.1 The Snapshot Assumption and Its Discontents 5

1.2 The Erasure of Causal Structure 6

1.3 The Irreversibility of Execution 6

1.4 Toward an Event-Historical Ontology 6

2 Histories as Primitive Computational Objects **7**

2.1 Events 7

2.2 Histories 7

2.3 Partial Ordering of Events 8

2.4 Derived State 8

3 The Minimal Operational Kernel **9**

3.1 Extension 9

3.2 Merge 10

3.3 Reduction 10

3.4 Kernel Sufficiency 11

4 The Algebra of Histories **11**

4.1 Prefix Ordering 11

4.2 Monotonicity of Extension 11

4.3 Merge as Join 12

4.4 Deterministic Replay 12

4.5 Reduction and Information Loss 13

4.6 The Monotone Potential 13

4.7 The Join-Semilattice and History Joins 13

5 Histories as an Order-Enriched Monoidal Category **14**

5.1 Dual Operational Representations 15

5.2 A Universal Property of the History Algebra 15

6 History Cuts and Factorization **16**

6.1 History Segments and Cuts 16

6.2 Reduction and Compositional Stability 16

6.3 Merge Consistency via Cuts 17

7 Dual Divisibility and the Direction of Execution **17**
7.1 Left and Right Divisibility 17
7.2 Asymmetry of Dual Representations 18
7.3 Structural Consequences for the Kernel 19

8 Merge Consistency and Reduction Stability **19**
8.1 Divisibility as a Merge Consistency Condition 19
8.2 Reduction and Compositional Preservation 20
8.3 Distributed Computation 20

9 Determinism and Replayability **21**
9.1 Replay and Determinism 21
9.2 State as Cache 21
9.3 Causal Correctness 21

10 Composition Through History Merge **22**
10.1 Divergent Branches and Reconciliation 22
10.2 Examples from Practice 22

11 Abstraction as Reduction **23**
11.1 Layered Reductions 23
11.2 Abstraction and Algorithmic Complexity 23

12 Time, Causality, and Irreversibility **23**
12.1 Two Modes of Irreversibility 24
12.2 Temporal Structure from Causal Order 24
12.3 Event Histories and Constraint Propagation in Physical Systems 24
12.4 Monotone Constraint Dynamics 25

13 Execution as Ordered History Geometry **25**

14 Unified Constraint-Driven Computation **26**
14.1 Deterministic Replay Systems 26
14.2 Distributed Convergence 27
14.3 Version-Control Systems 27
14.4 Constraint-Based Programming 27
14.5 Common Algebraic Structure 27

15 Conclusion **28**

Introduction

The dominant traditions of theoretical computer science describe programs in terms of transformations between states. From early automata theory to modern programming-language semantics, computational systems are modeled as structures that occupy well-defined configurations and evolve by transitioning among them. These models have proven extraordinarily productive for reasoning about correctness, complexity, and program behavior, furnishing the foundations of the discipline.

Yet the systems that computing actually relies upon increasingly strain the assumptions underlying state-centered models. Large-scale distributed services, append-only logs, version-control systems, and event-sourced architectures do not treat state as the primary object of computation. They record sequences of events whose cumulative effects determine the observable condition of the system. In such architectures, the present configuration is best understood not as a self-contained object but as a compressed summary of the history that produced it. The ordering of events, the causal dependencies among them, and the constraints propagated through their interactions carry information that cannot be recovered from a snapshot taken at any single moment.

This observation suggests a different foundational starting point. Rather than treating states as primitive and histories as auxiliary traces or mere byproducts of execution, one may reverse the relationship and regard histories as the fundamental structures from which states are derived. Such a reversal is not merely terminological; it reshapes the conceptual apparatus available for analyzing execution, composition, and abstraction. Computation, on this view, is not primarily state transformation but history construction. More precisely, computation emerges from the algebra of history composition.

The present paper develops a minimal operational framework built upon this reversal. In this framework, computation is interpreted as the irreversible construction of event histories under locally propagated constraints. System states emerge as reductions of those histories, and abstraction itself is understood as a form of controlled information compression applied to execution traces. The resulting kernel is minimal in the sense that three operations—extension, merge, and reduction—suffice to generate the full space of computationally relevant structures, and no proper subset of these operations is adequate to this task.

The argument proceeds as follows. Section 1 examines the structural limitations of state-centered ontologies of computation and motivates the transition to an event-historical perspective. Section 2 introduces event histories as the primitive objects of the theory and derives states as projections of those histories. Section 3 defines the minimal operational kernel and characterizes each of its three operations. Sections 9 through 11 develop the operational consequences of the kernel for determinism, composition, and abstraction respectively. Section 4 formalizes the algebraic structure induced by the kernel operations and proves the main structural theorems. Section 5

develops the categorical formulation of histories as an order-enriched monoidal category. Section 6 investigates history cuts and factorization. Section 7 establishes the dual divisibility results and their implications for the direction of execution. Section 8 connects divisibility to merge consistency and reduction stability. Section 12 investigates the emergence of temporal structure, causality, and irreversibility from within the framework. Section 14 traces the common algebraic pattern across a range of computational architectures and physical dynamical systems. Section 15 draws together the conclusions of the analysis.

1 The Failure of State Ontology

The choice of a foundational ontology for the theory of computation is not a purely formal matter. It conditions what can be expressed within the theory, what questions the theory is capable of posing, and what phenomena are visible within its conceptual framework. The state-centered ontology that has dominated computational theory since its inception carries specific assumptions that, while natural from one perspective, prove limiting when applied to the class of systems that contemporary computation relies upon.

1.1 The Snapshot Assumption and Its Discontents

The core commitment of state-based computation is what we may call the snapshot assumption: the thesis that the complete condition of a system at any moment can be captured by a snapshot—a self-contained, instantaneously observable configuration. Formal automata, transition systems, and denotational semantics all share this commitment. The state of a system is taken to be a well-defined element of some set S, and computation is modeled as a sequence of transitions

$$s_0 \xrightarrow{\tau_1} s_1 \xrightarrow{\tau_2} s_2 \xrightarrow{\tau_3} \cdots$$

in which each state s_i fully encodes the condition of the system at time i.

This picture is well-suited to sequential, isolated, and reversible computation. It captures finite automata, Turing machines, and the denotational semantics of pure functional programs with considerable clarity. The theoretical yield of this framework—decidability results, complexity classifications, program logics, type theories—has been immense.

What the snapshot assumption cannot easily accommodate, however, is the class of systems in which the present configuration is intelligible only in relation to the sequence of events that produced it. Distributed databases require that the history of transactions be preserved in order to maintain consistency guarantees across replicas. Version-control systems derive their entire meaning from the directed graph of commits that encodes the development of an artifact over time.

Event-sourced architectures store operations rather than states precisely because the sequence of operations carries information that the resulting state alone does not. In each of these cases, a snapshot may summarize the system, but it does not explain how the system came to be, nor does it preserve the information required to verify that its current configuration is legitimate.

1.2 The Erasure of Causal Structure

When computation is modeled purely as state transformation, the causal structure of execution is systematically erased. Two systems may arrive at observationally indistinguishable states through radically different histories. These distinct histories can have implications for correctness, safety, auditability, and reproducibility that are completely invisible to any analysis conducted at the level of states alone.

The state of a distributed ledger, for instance, cannot be validated without examining the history of transactions that produced it. The integrity of a filesystem cannot be guaranteed by inspecting a snapshot if the sequence of write operations is unavailable. The meaning of a version-controlled repository is not located in the current working tree but in the commit graph that generated it. In each of these domains, history is not auxiliary metadata appended to a primary state; it is the primary structure of the system, and the state is its derived projection.

1.3 The Irreversibility of Execution

Execution is fundamentally an irreversible process. Once an event has occurred—a message has been delivered, a transaction committed, a sensor reading recorded—that event cannot be undone without introducing additional compensating events that themselves become part of the system's history. This irreversibility is not a contingent feature of particular implementations; it reflects the temporal asymmetry of physical processes and of information-theoretic change more generally.

The state-transition model attempts to compress this irreversible process into a sequence of snapshots, treating each state as fully determined by its predecessor under the transition function. This compression becomes problematic in precisely those cases where the ordering of transitions, the concurrency among them, or the causal dependencies between them bear on the behavior of the system in ways that cannot be recovered from the current state.

1.4 Toward an Event-Historical Ontology

The analysis above motivates a reorientation of the foundational ontology. Rather than treating state as primitive and history as derived, we propose to treat history as primitive and state as derived.

Principle 1 (Event-Historical Primacy)**.** *The primary structure of a computational system is the irreversible history of events through which that system has been constructed. System states are projections of this history rather than primitive objects of the theory.*

This shift changes the unit of analysis from a configuration to a process of construction. It makes explicit what state-centered models obscure: that computation is a temporal, irreversible, and causally structured activity, not a timeless membership relation between configurations and transition functions.

2 Histories as Primitive Computational Objects

If computation is fundamentally the construction of irreversible histories, then the primitive object of the theory must be the history itself rather than the state it produces. This section introduces these objects with the precision required for the algebraic development that follows, while preserving the philosophical clarity that motivates the framework.

2.1 Events

The atom of computation in this framework is not a state or a configuration but an event: a record of an irreversible transition that has occurred within the system.

Definition 1 (Event)**.** *An* event *is an irreversible atomic transition in a computational system. Formally, events are elements of a set $\mathcal{E}$, called the* event universe*, whose elements represent the class of observable transitions available to the system under consideration.*

Events are intentionally abstract. Their internal structure—whether they encode messages, computations, sensor readings, or state mutations—is not fixed by the theory. What matters is not the content of events but their ordering relations and their effects on the causal structure of the histories they inhabit.

2.2 Histories

A history is a causally ordered accumulation of events. Its essential properties are monotonicity—events once appended cannot be removed—and causal structure—the ordering among events records the dependencies that governed their occurrence.

Definition 2 (Event History)**.** *An* event history H *is a finite or countably infinite sequence of events*

$$H = (e_1, e_2, e_3, \ldots)$$

such that each event e_{k+1} represents a valid extension of the history $(e_1, \ldots, e_k)$ determined by the system's transition rules.

The qualifier "valid extension" is crucial. Not every event may be appended to every history; the admissibility of an extension is governed by the causal and semantic constraints of the system. The validity condition encodes these dependencies and ensures that histories respect causal order.

Principle 2 (Irreversibility)**.** *If H is a history and e is an event appended to H, then the resulting history $H' = H \cdot e$ strictly extends H. Histories grow monotonically: events may be appended but not removed.*

2.3 Partial Ordering of Events

In many systems events are not totally ordered. Independent operations may proceed concurrently, giving rise to histories with a richer causal structure than a simple linear sequence.

Definition 3 (Causal Order)**.** *An* event history *may equivalently be represented as a pair $(H, \leq_H)$ in which H is a set of events and $\leq_H$ is a partial order over H such that $e_i \leq_H e_j$ if and only if e_i causally precedes e_j in the history.*

A totally ordered history is the special case in which $\leq_H$ is a total order. When events are concurrent—that is, when neither $e_i \leq_H e_j$ nor $e_j \leq_H e_i$—they may occur in either order without violating causal constraints.

Definition 4 (Concurrent Events)**.** *Events e_i and e_j are* concurrent *if neither $e_i \leq_H e_j$ nor $e_j \leq_H e_i$ holds within the history poset.*

Remark 1. *This formalization is closely related to Lamport's treatment of causality in distributed systems [12], wherein the causal precedence relation over events defines a consistent global ordering. The event-historical kernel generalizes this framework by making the causal structure of histories the primary object of analysis rather than a derived property of a state-based system.*

2.4 Derived State

With histories established as primary, states are introduced as derived projections.

Definition 5 (State Projection)**.** *A* state projection *is a deterministic function*

$$\sigma : \mathcal{H} \to S$$

that maps each admissible history to an element of a state space S*. The value $\sigma(H)$ is called the* state induced *by history H.*

Different projections may derive different aspects of the same history. The current contents of a memory region, the configuration of a filesystem, the balance of a transaction ledger, and the aggregate metrics reported by a monitoring system are all distinct projections applied to the same underlying event history. State is therefore not fundamental; it is a compressed view of execution history.

3 The Minimal Operational Kernel

Having established event histories as the primitive objects of the theory, we now identify the minimal set of operations sufficient to construct and transform those histories. Three operations constitute the kernel: *extension*, *merge*, and *reduction*. The space of executable histories is generated by these three primitive operations, and all computational processes arise as compositions of these generators.

3.1 Extension

Extension is the fundamental operation of execution. It takes an existing history and an event satisfying the system's admissibility conditions, and produces a new history strictly containing the original.

Definition 6 (History Extension)**.** *Let $H \in \mathcal{H}$ be an admissible history and let $e \in \mathcal{E}$ be an event such that appending e to H preserves the causal ordering and satisfies the system's validity conditions. The* extension operator *is a partial function*

$$\mathrm{ext} : \mathcal{H} \times \mathcal{E} \rightharpoonup \mathcal{H}$$

defined by $\mathrm{ext}(H, e) = H \cdot e$, where $H \cdot e$ denotes the history obtained by appending e to H as a maximal element under the causal order.

The partiality of ext is essential: not every event may be appended to every history. The domain of ext is exactly the set of pairs (H, e) for which the extension is causally admissible.

Principle 3 (Monotonic Growth)**.** *For any admissible pair (H, e), the extended history satisfies $H \preceq H \cdot e$ under the prefix ordering. Extension is a monotone operation with respect to causal containment.*

3.2 Merge

Merge realizes composition. When independent histories are to be combined—when two replicas synchronize, two branches reconcile, or two concurrent processes join—the resulting joint history must incorporate the events of both predecessors while preserving their respective causal orderings.

Definition 7 (History Merge)**.** *Let $H_1, H_2 \in \mathcal{H}$ be histories that share a common prefix $H_0 \preceq H_1$ and $H_0 \preceq H_2$. The histories are* compatible *if their events can be combined without violating causal constraints or system invariants. For compatible H_1 and H_2, the* merge operation *produces a history*

$$H_1 \sqcup H_2 \in \mathcal{H}$$

satisfying $H_1 \preceq H_1 \sqcup H_2$ and $H_2 \preceq H_1 \sqcup H_2$, and minimal with this property: for any H with $H_1 \preceq H$ and $H_2 \preceq H$, one has $H_1 \sqcup H_2 \preceq H$.

The minimality condition ensures that the merged history adds only what is necessary to subsume both branches. It corresponds precisely to the least upper bound, or join, in a partially ordered set, and it is this correspondence that gives the space of histories its semilattice structure.

When histories are not directly compatible, merge requires the introduction of compensating events that restore consistency. These compensating events are themselves recorded in the merged history, preserving irreversibility while explicitly encoding the reconciliation process. Conflict resolution is not erasure; it is history extension by another means.

3.3 Reduction

Reduction is the operation by which abstraction, summarization, and state derivation are realized. It maps a history to a compressed representation that preserves selected structural properties while discarding others.

Definition 8 (History Reduction)**.** *A* reduction operator *is a function*

$$\rho : \mathcal{H} \to R$$

where R is a space of reduced representations. The operator ρ preserves *a property P of histories if whenever H possesses P, $\rho(H)$ possesses the corresponding property in R.*

Reductions include the construction of snapshots, the generation of database views, the formation of audit logs, and any other operation that derives a simplified description from a full execution trace. The key observation is that reductions do not generate new computational structure; they remove information from existing histories by collapsing multiple distinct histories into a single representation.

Principle 4 (Abstraction as Reduction)**.** *An abstraction is a reduction of an event history that preserves only the distinctions relevant to a particular task. Abstraction is not the creation of higher-level objects but the controlled loss of information.*

3.4 Kernel Sufficiency

The three operations of extension, merge, and reduction are jointly sufficient to generate the space of computational structures considered in this framework. Extension constructs histories through execution. Merge composes histories through reconciliation. Reduction derives representations through compression. From these generators arise deterministic replay, distributed consensus, versioned data structures, and reproducible computation.

4 The Algebra of Histories

The three operations of the minimal kernel induce a natural algebraic structure over the space of event histories. We develop this structure here, establishing what emerges when extension, merge, and reduction are taken seriously as organizing principles.

4.1 Prefix Ordering

Histories are ordered by extension.

Definition 9 (Prefix Order)**.** *The* prefix order *on $\mathcal{H}$ is defined by*

$$H_1 \preceq H_2 \iff H_1 \text{ is a prefix of } H_2,$$

that is, H_1 is a causally embedded initial segment of H_2 under the causal ordering of H_2.

Proposition 1. *The pair $(\mathcal{H}, \preceq)$ is a partially ordered set.*

Proof. Reflexivity holds because every history is a prefix of itself. Antisymmetry holds because if $H_1 \preceq H_2$ and $H_2 \preceq H_1$, then H_1 and H_2 are mutually initial segments of each other, which forces $H_1 = H_2$. Transitivity holds because if H_1 is a prefix of H_2 and H_2 is a prefix of H_3, then H_1 is a prefix of H_3. □

4.2 Monotonicity of Extension

Theorem 1 (Monotonic Extension)**.** *For any history $H \in \mathcal{H}$ and any event $e \in \mathcal{E}$ such that $\mathrm{ext}(H, e)$ is defined,*

$$H \preceq \mathrm{ext}(H, e).$$

Proof. By definition, $\mathrm{ext}(H, e) = H \cdot e$ is the history obtained by appending e to H as a maximal element under the causal order. The events of H constitute an initial segment of $H \cdot e$ under this order. Hence H is a prefix of $H \cdot e$, giving $H \preceq H \cdot e$. □

Corollary 1. *The extension operator is strictly monotone on non-trivially extended histories: if* $\mathrm{ext}(H, e)$ *is defined, then* $H \prec \mathrm{ext}(H, e)$.

4.3 Merge as Join

Theorem 2 (Merge Convergence)**.** *Let* $H_1, H_2 \in \mathcal{H}$ *be compatible histories. If the merge* $H_1 \sqcup H_2$ *exists in* $\mathcal{H}$*, then it is the least upper bound of* $\{H_1, H_2\}$ *under the prefix order.*

Proof. By definition of merge, $H_1 \preceq H_1 \sqcup H_2$ and $H_2 \preceq H_1 \sqcup H_2$, so $H_1 \sqcup H_2$ is an upper bound. Suppose $H \in \mathcal{H}$ satisfies $H_1 \preceq H$ and $H_2 \preceq H$. Then H contains all events of H_1 and H_2 as substructures respecting the causal ordering of each. By the minimality condition in the definition of merge, $H_1 \sqcup H_2$ is the smallest history with this property, hence $H_1 \sqcup H_2 \preceq H$. □

Proposition 2. *When merge is defined for all compatible pairs in* $\mathcal{H}$ *and the compatibility relation is closed under iteration, the space* $(\mathcal{H}, \preceq, \sqcup)$ *forms a join-semilattice on compatible subsets of* $\mathcal{H}$*.*

Proof. Commutativity follows from the symmetric treatment of H_1 and H_2: the least upper bound of $\{H_1, H_2\}$ equals that of $\{H_2, H_1\}$. Idempotency follows because the least upper bound of $\{H, H\}$ is H itself. Associativity follows because the least upper bound of $\{H_1 \sqcup H_2, H_3\}$ is the least history containing H_1, H_2, and H_3, which equals the least upper bound of $\{H_1, H_2 \sqcup H_3\}$. □

4.4 Deterministic Replay

Theorem 3 (Deterministic Replay)**.** *Let* $\sigma : \mathcal{H} \to S$ *be a reduction operator and suppose the semantics of each event are deterministic. Then* $\sigma(H)$ *is uniquely determined by* H.

Proof. Replay constructs $\sigma(H)$ by applying the events of H in their causal order to an initial configuration. Since the causal order is a partial order, any two causal linearizations of H produce the same final configuration when event semantics are deterministic. This follows from the Church–Rosser property of deterministic rewriting systems: if the order of application does not affect the result of each deterministic step, then all causal linearizations produce the same outcome. Hence $\sigma(H)$ depends only on H. □

4.5 Reduction and Information Loss

Principle 5 (Irreversibility of Reduction)**.** *For a reduction operator $\rho : \mathcal{H} \to R$, it is in general the case that*

$$\rho(H_1) = \rho(H_2) \not\Rightarrow H_1 = H_2.$$

Distinct histories may collapse to the same reduced representation under ρ.

Proposition 3 (Non-Injectivity of Reduction)**.** *For any non-trivial reduction operator $\rho : \mathcal{H} \to R$, there exist histories $H_1, H_2 \in \mathcal{H}$ with $H_1 \neq H_2$ and $\rho(H_1) = \rho(H_2)$.*

Proof. A trivial reduction would be the identity on $\mathcal{H}$, which retains all information. Any non-trivial reduction ρ discards at least one bit of distinguishing information between histories; hence there exist distinct histories that map to the same element of R. If ρ were injective on all of $\mathcal{H}$, it would constitute a recoding rather than a compression. □

4.6 The Monotone Potential

To characterize execution dynamics within the history algebra, we introduce a potential function that measures the degree to which a history satisfies the system's constraints.

Definition 10 (Monotone Potential)**.** *A* monotone potential *is a function $E : \mathcal{H} \to \mathbb{R}$ such that*

$$H_1 \preceq H_2 \Rightarrow E(H_2) \leq E(H_1).$$

Execution proceeds in the direction of non-increasing potential.

Definition 11 (Stable History)**.** *A history $H \in \mathcal{H}$ is* stable *if no valid extension $H' = \mathrm{ext}(H, e)$ satisfies $E(H') < E(H)$. Stable histories are fixed points of the descent dynamics.*

The entropy-like quantity measuring macroscopic indeterminacy of a reduced representation r is given by

$$S(r) = \log \Omega(\rho, r) = \log \left| \{H \in \mathcal{H} : \rho(H) = r\} \right|.$$

This quantity grows as reduction discards finer detail, in exact analogy with the Boltzmann entropy of a thermodynamic macrostate.

4.7 The Join-Semilattice and History Joins

Definition 12 (History Join)**.** *For compatible histories H_1 and H_2, the join*

$$H_1 \vee H_2$$

is the least history containing both as prefixes.

Proposition 4. *If merge operations are associative, commutative, and idempotent, then* $(\mathcal{H}, \vee)$ *forms a join-semilattice.*

These properties appear naturally in systems where histories accumulate monotonically and merges preserve causal structure. The resulting space of histories forms a structured space of executions, and the derived structure—poset, semilattice, potential landscape—is not imposed externally but emerges from the three kernel operations alone.

5 Histories as an Order-Enriched Monoidal Category

The structures introduced throughout this paper admit a concise mathematical formulation that clarifies the deeper nature of the history algebra.

Let $\mathcal{H}$ denote the category of executable histories. Objects represent system interfaces or boundary conditions, while morphisms represent history segments connecting those boundaries. Composition of morphisms corresponds to temporal concatenation of histories:

$$h_{u \to t} = h_{s \to t} \circ h_{u \to s}.$$

In addition to composition, histories support a parallel combination operation. Independent histories may be executed simultaneously, giving rise to a monoidal product

$$h_1 \otimes h_2.$$

Together these operations endow $\mathcal{H}$ with the structure of a monoidal category.

However, histories also carry an intrinsic informational ordering. A history that records more events excludes more alternatives and therefore contains strictly more information about the execution. This naturally induces a partial order

$$h_1 \preceq h_2$$

whenever h_2 refines h_1 by recording additional events.

Composition and parallel combination are monotone with respect to this ordering. If $h_1 \preceq h_2$ then

$$f \circ h_1 \preceq f \circ h_2 \qquad \text{and} \qquad h_1 \otimes g \preceq h_2 \otimes g$$

whenever the operations are defined. Consequently the category of histories is naturally enriched over ordered sets: each hom-set $\mathcal{H}(x, y)$ forms a partially ordered set describing the refinement

structure of histories between those boundaries.

Within this framework the primitive operations of the minimal operational kernel acquire clear structural interpretations. Extension corresponds to composition with additional history segments. Merge corresponds to identifying compatible histories sharing common prefixes. Reduction corresponds to order-preserving maps that collapse distinctions between histories.

Admissibility constraints restrict the morphisms that may appear in valid executions. In many settings these constraints form a downward-closed subset of the hom-set order. The resulting structure describes the space of lawful histories that a system may realize.

5.1 Dual Operational Representations

This categorical perspective clarifies why the direction of execution matters. Representations of histories as state transformations and as observable transformations correspond to functors

$$F : \mathcal{H} \to \mathbf{OrdMon}, \qquad G : \mathcal{H}^{op} \to \mathbf{OrdMon}.$$

Although these representations are prediction-equivalent, they may induce different admissibility constraints on intermediate history segments. As a result, the factorization properties of histories can depend on whether execution is evaluated through forward extension or through its dual representation.

The algebra of histories therefore forms an order-enriched monoidal structure whose operational interpretations need not be self-dual. This structure provides the mathematical foundation for the analysis of divisibility developed in the sections that follow.

5.2 A Universal Property of the History Algebra

The operations of extension, merge, and reduction generate the space of executable histories in a precise sense. Let $\mathcal{H}$ denote the algebra of histories generated by these operations, subject to the compositional relations introduced above. An execution in any concrete computational system may then be interpreted as a structure-preserving map from $\mathcal{H}$ into the operational algebra of that system.

Proposition 5 (Universal property of the history algebra)**.** *The history algebra $\mathcal{H}$ is initial among computational structures equipped with extension, merge, and reduction operations. That is, for any such structure $\mathcal{A}$ there exists a unique structure-preserving map*

$$\Phi : \mathcal{H} \to \mathcal{A}.$$

Intuitively, this property states that $\mathcal{H}$ contains no structure beyond that required by the minimal

operational kernel. Any concrete model of computation implementing these operations factors through the history algebra. In this sense, the algebra of histories plays a role analogous to that of a free algebra generated by a set of primitive operations: the extension–merge–reduction kernel provides a universal description of history-based computation.

6 History Cuts and Factorization

The order-enriched monoidal structure introduced in the previous section raises a fundamental question: does every history admit a lawful factorization through intermediate cuts? In ordinary state-transition models this question rarely appears, because transitions are defined directly between states. In a history framework it becomes central.

6.1 History Segments and Cuts

Consider a history segment

$$h_{0\to t}.$$

A *cut* at time s with $0 \le s \le t$ partitions this history into two segments

$$h_{0\to s}, \qquad h_{s\to t},$$

such that $h_{0\to t} = h_{s\to t} \circ h_{0\to s}$.

For the existence of lawful intermediate execution steps to be guaranteed, one requires that for every cut s, the segment $h_{s\to t}$ completing the history from that cut is itself admissible. This is the condition of divisibility.

Definition 13 (Divisible History)**.** *A history $h_{0\to t}$ is* divisible *if for every s with $0 \le s \le t$, the history admits a decomposition $h_{0\to t} = h_{s\to t} \circ h_{0\to s}$ in which both segments are admissible histories.*

Divisibility is not automatic. It is a structural property of the history that depends on the admissibility constraints of the system.

6.2 Reduction and Compositional Stability

For reductions to preserve executability, they must respect the factorization structure of histories. Whenever a history admits a valid decomposition

$$h_{0\to t} = h_{s\to t} \circ h_{0\to s},$$

the reduced histories must admit a compatible decomposition

$$R(h_{0\to t}) = R(h_{s\to t}) \circ R(h_{0\to s}).$$

If this property fails, reduction destroys information necessary for lawful extension and merge operations. A reduction operator that violates this condition cannot be used as a safe abstraction within the kernel.

6.3 Merge Consistency via Cuts

Merge operations impose a closely related requirement. Suppose two histories share a common prefix $h_{0\to s}$ and we attempt to merge their continuations

$$h_{0\to t_1}, \qquad h_{0\to t_2}$$

into a combined execution structure. Such a merge is admissible only when the intermediate extensions at s remain compatible within the ordered structure of histories. Divisibility therefore acts as a consistency condition for merge: if intermediate extensions fail to remain admissible at every cut, histories cannot be safely merged without introducing violations of the execution constraints.

7 Dual Divisibility and the Direction of Execution

A subtle but fundamental structural feature of history-based computation emerges from the categorical analysis: the two natural dual representations of histories as state transformations and observable transformations impose asymmetric divisibility constraints. This asymmetry reveals that execution possesses an intrinsic direction not immediately visible in the operational description.

7.1 Left and Right Divisibility

Working within the category $\mathcal{H}$, we ask whether every history can be factorized through arbitrary intermediate cuts under each of the two dual representations.

Definition 14 (Left divisibility)**.** *A history $h_{0\to t}$ is* left divisible *if for every cut $0 \le s \le t$ there exists an admissible history segment $k_{s\to t}$ such that*

$$h_{0\to t} = k_{s\to t} \circ h_{0\to s}.$$

Intuitively, left divisibility states that a global execution can be reconstructed by extending any valid prefix with a lawful continuation. This corresponds directly to the operational meaning of extension in the minimal kernel.

Definition 15 (Right divisibility)**.** *A history $h_{0\to t}$ is* right divisible *if for every cut $0 \le s \le t$ there exists an admissible history segment $r_{0\to s}$ such that*

$$h_{0\to t} = h_{s\to t} \circ r_{0\to s}.$$

Although these definitions appear symmetric, they need not coincide. The distinction becomes visible whenever admissibility constraints are defined relative to an ordered structure over histories.

7.2 Asymmetry of Dual Representations

Theorem 4 (Dual divisibility of operational representations)**.** *Let histories be represented operationally in two dual ways:*

$$F : \mathcal{H} \to \mathbf{OrdMon}, \qquad G : \mathcal{H}^{op} \to \mathbf{OrdMon},$$

where F acts on states and G acts on effects. Suppose the two representations are prediction-equivalent in the sense that

$$\langle F(h)(\rho), A\rangle = \langle \rho, G(h)(A)\rangle$$

for all states ρ, effects A, and histories h.

Then left divisibility of histories relative to the state order does not in general imply right divisibility relative to the effect order.

This asymmetry arises because the admissibility cones governing state transformations and observable transformations differ. Operationally, the distinction corresponds to two different ways of generating histories in continuous time. One obtains either

$$\dot{\Phi}_t = L_t \circ \Phi_t \qquad \text{or} \qquad \dot{\Phi}_t = \Phi_t \circ R_t,$$

corresponding respectively to postcomposition and precomposition of infinitesimal execution steps. The connection to the Schrödinger and Heisenberg pictures in quantum dynamics is direct: the former evolves states forward while keeping observables fixed; the latter propagates observables backward while keeping states fixed [19, 3, 18].

7.3 Structural Consequences for the Kernel

Consequently, execution histories possess an intrinsic directional structure. Extension corresponds to lawful postcomposition of history segments, while the dual observable representation induces a precomposition dynamics. These two execution monoids are prediction-equivalent but structurally distinct.

In the language of the minimal operational kernel, this observation reveals that the extension operation is fundamentally asymmetric. The admissibility of intermediate execution steps depends on the direction in which history factorization is evaluated. Merge and reduction therefore operate over histories whose compositional structure is not strictly self-dual. The direction in which execution is evaluated—whether through forward extension of states or backward propagation of observables—affects the existence of lawful intermediate steps.

8 Merge Consistency and Reduction Stability

The divisibility properties established in the previous section have direct operational implications for the history-based model of computation. In particular, they determine whether histories can be safely partitioned, merged, and reduced without violating the admissibility constraints of the execution kernel.

8.1 Divisibility as a Merge Consistency Condition

If the history $h_{0\to t}$ is left divisible, there exists an admissible continuation map $k_{s\to t}$ such that

$$h_{0\to t} = k_{s\to t} \circ h_{0\to s}.$$

Operationally, this guarantees that any valid prefix can be extended without ambiguity: the prefix contains sufficient information for lawful continuation of the computation.

Merge operations impose a stronger requirement. Suppose two histories share a common prefix $h_{0\to s}$. We may attempt to merge their continuations,

$$h_{0\to t_1}, \qquad h_{0\to t_2},$$

into a combined execution structure. Such a merge is admissible only when the intermediate extensions remain compatible within the ordered structure of histories.

Divisibility therefore acts as a consistency condition for merge. If intermediate extensions fail to remain admissible at every cut, histories cannot be safely merged without introducing violations

of the execution constraints.

8.2 Reduction and Compositional Preservation

Reduction operations introduce a complementary condition. A reduction map

$$R : \mathcal{H} \to \widetilde{\mathcal{H}}$$

removes distinctions between histories by collapsing them into coarser equivalence classes. For reductions to preserve executability, they must respect the factorization structure of histories: whenever a history admits a valid decomposition

$$h_{0\to t} = h_{s\to t} \circ h_{0\to s},$$

the reduced histories must admit a compatible decomposition

$$R(h_{0\to t}) = R(h_{s\to t}) \circ R(h_{0\to s}).$$

If this property fails, reduction destroys information necessary for lawful extension and merge operations, and the resulting reduced histories cannot serve as safe abstractions within the operational kernel.

8.3 Distributed Computation

This connects divisibility to the requirements of distributed computation. In distributed systems, concurrent processes produce independent histories that must later be reconciled through merge. The safety of such reconciliation depends on whether each process's history remains divisible at the points where the merge is to be performed. A failure of divisibility at an intermediate cut means that the system cannot guarantee lawful continuation after reconciliation, threatening the consistency guarantees that make distributed computation tractable.

Taken together, divisibility, merge consistency, and reduction stability constitute the structural compatibility conditions for the minimal operational kernel. Extension guarantees the existence of lawful continuations; merge guarantees compatibility between concurrent histories; and reduction guarantees that coarse-grained representations preserve the compositional structure of execution. These three conditions define the domain in which history-based computation remains stable under composition and abstraction.

9 Determinism and Replayability

Within the event-historical framework, determinism must be reformulated. The classical notion—that a deterministic system always produces the same output given the same input—is stated at the level of states and transitions. Transposed into the language of histories, determinism becomes a property of replay rather than of transition.

9.1 Replay and Determinism

Definition 16 (Replay)**.** *Given a history $H = (e_1, e_2, \ldots, e_n)$ and a reduction operator $\sigma : \mathcal{H} \to S$, the* replay *of H under σ is the sequential application of the events $(e_1, \ldots, e_n)$ to an initial configuration, producing the derived state $\sigma(H) \in S$.*

Definition 17 (Replay Determinism)**.** *A system with reduction operator σ is* replay-deterministic *if $\sigma(H)$ is uniquely determined by H; that is, if the mapping $\sigma : \mathcal{H} \to S$ is a well-defined function rather than a relation.*

Replay determinism is a weaker and more operationally grounded condition than classical state determinism. It does not require that the system produce identical event sequences when run repeatedly; what it requires is that any given history, when replayed, produces a unique derived state. This is the condition that makes a history an authoritative record of execution.

9.2 State as Cache

Within the event-historical framework, the state of a system is best understood as a cache derived from history. A snapshot represents the value of $\sigma(H)$ at some moment, stored to avoid the cost of replaying the full history. The snapshot accelerates access to derived information but does not replace or supersede the history.

If a cached state is lost—through a crash, a hardware fault, or a software error—it can be reconstructed by replaying the underlying history. The history is the authoritative source; the cache is a performance optimization. This architecture underlies distributed logging systems, database write-ahead logs, and event-sourced applications [8, 11].

9.3 Causal Correctness

Principle 6 (Causal Correctness)**.** *A derived state $s \in S$ is* causally correct *if there exists an admissible history $H \in \mathcal{H}$ such that $\sigma(H) = s$. A state that cannot be produced by replaying any admissible history is causally incorrect regardless of its syntactic validity.*

Causal correctness cannot be verified from the state alone; it requires access to the history from which the state was derived. This observation is especially significant in domains such as distributed ledgers, cryptographic audit logs, and fault-tolerant replicated state machines.

10 Composition Through History Merge

In classical programming models, composition is understood as a structural operation: programs are combined by placing them in sequential, parallel, or hierarchical arrangements. Within the event-historical framework, composition is instead understood as a historical operation: programs compose when their execution histories are merged into a common continuation.

10.1 Divergent Branches and Reconciliation

Execution frequently gives rise to divergent histories. Let H_0 be a common prefix, and suppose two independent lines of execution extend it:

$$H_1 = H_0 \cdot A, \qquad H_2 = H_0 \cdot B,$$

where $A = (a_1, \ldots, a_m)$ and $B = (b_1, \ldots, b_n)$ are sequences of events extending H_0 along independent causal paths. These histories represent two branches of execution whose events are concurrent: neither branch's events causally precede the other's.

Composition arises when these branches are reconciled. The merged history $H_3 = H_1 \sqcup H_2$ is a joint continuation that preserves the causal ordering within each branch while extending the common prefix H_0.

Principle 7 (Causal Preservation under Merge)**.** *For any events e_i, e_j with $e_i \leq_{H_1} e_j$ in H_1, the merged history $H_1 \sqcup H_2$ satisfies $e_i \leq_{H_1 \sqcup H_2} e_j$. The same holds for causal ordering within H_2.*

10.2 Examples from Practice

Distributed version-control systems such as Git organize repository evolution as a directed acyclic graph of commits, in which each merge commit incorporates the events of two or more branches while preserving their internal ordering [5]. Conflict-free replicated data types [20] achieve eventual consistency across distributed replicas precisely because their update operations form a join-semilattice. Event-sourced architectures [8] record all operations as append-only log entries and derive application state by replaying these logs. In each of these domains, the operational pattern is the same: independent histories are produced by parallel activity and subsequently reconciled through merge operations that preserve causal structure.

11 Abstraction as Reduction

If execution constructs histories and composition merges them, then abstraction must be understood as an operation on those histories. Traditional programming theory often treats abstraction as generative: types, classes, modules, and interfaces introduce new conceptual entities. Within the event-historical framework, abstraction is not generative. It is reductive. Abstraction discards distinctions in a history while preserving those relevant to a given purpose.

11.1 Layered Reductions

Multiple distinct reduction operators may be applied to the same underlying history. A single execution history within a computational environment may simultaneously support a filesystem snapshot, a database view, a performance monitoring summary, a security audit log, and a billing record. Each representation is derived from the same history by a distinct reduction that selects and preserves different aspects of the execution trace.

These derived structures should not be understood as independent realities. They are different informational projections of a single evolving history, reflecting the existence of multiple purposes each defining a different reduction over the same underlying events.

11.2 Abstraction and Algorithmic Complexity

The relationship between abstraction and information loss can be made precise through algorithmic complexity. For a finite history H, define its *Kolmogorov complexity* as

$$K(H) = \min\{|p| : U(p) = H\},$$

where U is a fixed universal Turing machine and $|p|$ denotes the length of program p. The quantity $K(H)$ measures the minimal description length from which the history can be reconstructed [13].

A reduction operator $\rho : \mathcal{H} \to R$ can be interpreted as a constrained compression: it maps H to a description $\rho(H)$ that is shorter than H but preserves selected observables. Unlike Kolmogorov-optimal compression, a reduction is designed to preserve specific properties rather than to minimize description length globally. It is therefore a *semantically constrained compression*.

12 Time, Causality, and Irreversibility

The event-historical framework does not presuppose time as a primitive dimension; rather, temporal structure emerges from within the framework as a property of the ordering relation over histories.

12.1 Two Modes of Irreversibility

The framework distinguishes two structurally distinct forms of irreversibility that operate simultaneously and reinforce each other.

The first is *execution irreversibility*: histories grow monotonically under extension, and events once incorporated cannot be removed without introducing compensating events that become part of the extended record. This irreversibility is formal: it follows directly from Theorem 1.

The second is *abstraction irreversibility*: reductions discard information about the causal ordering and intermediate structure of histories, and this discarded information cannot in general be recovered from the reduced representation alone. These two forms of irreversibility interact in a manner that closely parallels the relationship between microscopic dynamics and macroscopic thermodynamics.

12.2 Temporal Structure from Causal Order

The prefix ordering $\preceq$ on $\mathcal{H}$ induces a natural temporal direction. If $H_1 \prec H_2$, then H_2 is the later history: it contains all of H_1's events together with additional events that post-date them. The arrow of execution corresponds to monotone growth of causal structure; attempts to "reverse" execution do not undo events but rather select an earlier prefix and re-execute subsequent computation from that point.

12.3 Event Histories and Constraint Propagation in Physical Systems

The structural correspondence between the event-historical kernel and physical systems governed by local constraint propagation can be made precise through the Ising model. In this model, a lattice of N sites carries spin variables $s_i \in \{-1, +1\}$, and the system's energy is given by

$$E(\mathbf{s}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i,$$

where J is the spin-coupling constant, h is an external field, and $\langle i, j \rangle$ ranges over neighboring pairs [10].

Each spin flip at site i constitutes an event $e_k = (i_k, \Delta s_{i_k})$ in the kernel's event universe. The sequence of spin flips forms an event history $H = (e_1, e_2, \ldots, e_n)$, and the instantaneous spin configuration $\mathbf{s}$ is a state projection $\sigma(H) = \mathbf{s}$ derived from this history by replay. The energy $E(\mathbf{s})$ defines a monotone potential: Metropolis or Glauber dynamics accept transitions that reduce energy, and the system descends toward stable configurations.

This correspondence is not an analogy but a structural identity: Ising dynamics and constraint-driven event-historical computation are instances of the same abstract process—the monotonic extension of a causally ordered history under a locally propagated potential [4, 15].

12.4 Monotone Constraint Dynamics

Let $\mathcal{C}(H)$ denote the constraint set induced by history H. The extension of H by an event e updates this constraint set according to a local update function F:

$$\mathcal{C}(H \cdot e) = F(\mathcal{C}(H), e).$$

Principle 8 (Constraint Monotonicity)**.** *For histories $H_1 \preceq H_2$, the constraint sets satisfy $\mathcal{C}(H_1) \subseteq \mathcal{C}(H_2)$.*

Constraint monotonicity implies that execution does not remove previously established dependencies; it only introduces new ones. Combined with the monotone potential, this gives rise to descent dynamics in which each local event guides the history toward stable configurations in which all local constraints are mutually satisfied.

13 Execution as Ordered History Geometry

The central claim of this paper is that computation is most naturally understood not as the transformation of states but as the construction and manipulation of histories.

A history records the irreversible accumulation of events. Each event excludes alternative possibilities and therefore refines the set of admissible continuations. Computation emerges from the lawful composition of such exclusions.

Within this perspective, the minimal operational kernel consists of three primitive operations. Extension adds new events to an existing history, producing longer executions. Merge combines compatible histories by identifying shared prefixes and reconciling their subsequent developments. Reduction removes distinctions between histories by collapsing them into coarser summaries.

These operations generate an ordered structure over histories. The order captures informational refinement: a history that records more events contains strictly more constraints on the future. Composition of histories corresponds to concatenation of exclusions, while merge and reduction act as structural transformations within this ordered space.

The divisibility results discussed earlier illuminate a subtle property of this geometry. When histories are represented operationally as transformations acting on states, one obtains a particular notion of lawful intermediate execution. When the same histories are represented as transformations

acting on observables, a dual notion emerges. The two representations agree on empirical predictions but impose different constraints on how histories may be factorized.

This reveals that execution possesses an intrinsic directionality. The continuation of a history through extension is not equivalent to the reconstruction of a history from its future boundary. In categorical terms, histories admit two dual operational representations related by opposition of composition order. The admissibility constraints governing these representations need not coincide.

Seen from this perspective, computation resembles a geometric process in an ordered space of histories. Extension traces directed paths through this space, merge identifies compatible trajectories, and reduction projects trajectories onto coarser descriptions. The structure that emerges is closer to a geometry of execution than to a traditional state-transition system.

Many familiar computational architectures implicitly operate within this geometry. Distributed version control systems construct histories through successive extensions and reconcile them through merge operations. Event-sourced systems record append-only histories and derive system state through reduction. Collaborative editing systems maintain partially ordered logs of operations whose reconciliation depends on compatibility conditions between histories.

The minimal operational kernel proposed here abstracts these patterns into a general model of computation grounded in history composition. Within this model, the existence of lawful intermediate histories becomes a central structural property—and the divisibility phenomena observed across computational and physical systems appear not as isolated curiosities but as manifestations of a deeper algebraic feature of execution itself. Computation may be understood as the geometry of irreversible histories evolving within an ordered space of possibilities.

14 Unified Constraint-Driven Computation

The minimal operational kernel is not a model of any particular computational paradigm. It is an abstract characterization of a structural pattern that appears across multiple domains.

14.1 Deterministic Replay Systems

Event-sourced architectures [8, 11] are the most direct computational instantiation of the kernel. The primary persistent datum is not a mutable application state but an append-only log of events. Each log entry is an event in the kernel's sense; the cumulative log is the event history H; and the application state is the projection $\sigma(H)$ obtained by replaying the log. Checkpoints are reduction operators that compress the history to accelerate state reconstruction, while the log remains the authoritative source. Failures are recovered by replaying the log from a checkpoint, corresponding exactly to the causal correctness principle.

14.2 Distributed Convergence

Conflict-free replicated data types [20] achieve eventual consistency without centralized coordination by ensuring that local update operations form a join-semilattice. Each replica maintains an operation history growing monotonically over time. Synchronization proceeds through merge operations that combine histories while preserving causal ordering. Because merge is associative, commutative, and idempotent—the properties established in Proposition 2—repeated synchronization among any collection of replicas converges to a unique consistent merged history regardless of the order of synchronizations. Consistency is therefore a structural consequence of the algebra of histories rather than a property imposed by a coordination protocol.

14.3 Version-Control Systems

Distributed version-control systems such as Git [5] realize the history algebra with particular clarity. A repository is organized as a directed acyclic graph of commits, where each commit is an event extending the project's history. Branching corresponds to the divergence of compatible histories from a common prefix, and merge commits implement the join operation. The repository is therefore not merely a store of code revisions; it is a concrete instantiation of the event-historical algebra.

14.4 Constraint-Based Programming

Constraint-based programming systems compute solutions by propagating constraints through networks of variables. Within the event-historical framework, each propagation step is an event extending the computation history; the constraint set $\mathcal{C}(H)$ evolves monotonically; and the solution corresponds to a stable history in the sense of Section 4. The computation is not a search through a state space but a trajectory through the lattice of histories guided by the monotone potential defined by the constraint satisfaction degree.

14.5 Common Algebraic Structure

Despite their apparent diversity, these systems share the same abstract organization. In each, evolution proceeds through the monotonic extension of a causally ordered history. Composition arises through join-like merge operations that preserve causal ordering. Derived representations emerge through reductions that compress histories into purpose-specific projections. Stable configurations appear as fixed points of the descent dynamics induced by the monotone potential.

The convergence of these structural patterns across domains as different as distributed systems, software development tools, and statistical mechanics suggests that the event-historical kernel captures a genuine organizing principle. The organizational unity underlying distributed logs,

version graphs, constraint solvers, and physical lattice dynamics is not coincidental; it reflects the common mathematical structure of the history algebra developed in the preceding sections.

15 Conclusion

This paper has proposed and developed a minimal operational kernel for the interpretation of computation in which the fundamental structure of a system is not a static state but the irreversible history of events through which the system has been constructed. Traditional models of computation have generally described programs as transformations between well-defined states. While this perspective has yielded powerful formal tools, it necessarily compresses the temporal and causal structure of execution into a sequence of abstract snapshots—obscuring the processes by which systems actually evolve and removing from view the irreversible accumulation of events that produces observable behavior.

The framework developed here reverses this perspective by treating event histories as the primary objects of computation. Execution is understood as the monotonic extension of histories through the occurrence of events. Composition arises through the reconciliation and merging of compatible histories while preserving their causal ordering relations. Abstraction appears as the reduction of histories through controlled information loss. These operations together form a minimal operational kernel sufficient to generate the structures appearing across contemporary computational systems.

The algebra induced by these operations forms a join-semilattice over a partially ordered space of histories admitting a monotone potential whose descent characterizes execution dynamics. Three principal theorems govern the framework: the monotonicity of extension (Theorem 1), the convergence of merge to a least upper bound (Theorem 2), and the uniqueness of replay under deterministic event semantics (Theorem 3). These are structural consequences of the history algebra itself, not independent engineering desiderata.

The categorical formulation reveals that the algebra of histories is an order-enriched monoidal structure. The two natural dual representations—acting on states and observables respectively—are prediction-equivalent but impose asymmetric admissibility constraints on history factorization. This dual divisibility result exposes an intrinsic directionality in execution: the direction in which histories are evaluated, whether through forward extension of states or backward propagation of observables, affects the existence of lawful intermediate steps. The minimal operational kernel is therefore not self-dual, and this asymmetry is not a defect but a structural feature that distinguishes history-based computation from purely reversible models.

The framework's implications extend beyond a reformulation of programming semantics. The event-historical algebra appears, under appropriate interpretation, in event-sourced architectures, conflict-free replicated data types, distributed version-control systems, constraint-based program-

ming systems, and physical lattice models governed by local interaction potentials. In each of these domains, system behavior emerges from the monotonic accumulation of locally constrained events, composition is realized through join-like reconciliation of divergent histories, and abstraction is achieved through reduction operators that compress execution traces into purpose-specific projections.

The deeper significance of this unification is philosophical as much as technical. It reframes computation not as the traversal of a pre-existing state space but as an act of construction: the progressive building of irreversible causal structures through locally constrained transitions. State ceases to be the foundational object of the theory and becomes a derived convenience—a compressed view of the history from which it was produced. The temporal asymmetry of computation ceases to be an assumption imported from physics and becomes instead a structural theorem, a consequence of the irreversible extension operator that defines what it means to execute. Computation, in the end, is the geometry of irreversible histories evolving within an ordered space of possibilities.

References

[1] Samson Abramsky. *Proofs as Processes*. Theoretical Computer Science, 135(1):5–9, 1994.

[2] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008.

[3] Heinz-Peter Breuer, Elsi-Maria Laine, and Jyrki Piilo. *Measure for the Degree of Non-Markovian Behavior of Quantum Processes in Open Systems*. Physical Review Letters, 103(21):210401, 2009.

[4] Stephen G. Brush. *History of the Lenz–Ising Model*. Reviews of Modern Physics, 39(4):883–893, 1967.

[5] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2014.

[6] Thomas Cover and Joy Thomas. *Elements of Information Theory*. Wiley, 1991.

[7] Brian Davey and Hilary Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2002.

[8] Martin Fowler. *Event Sourcing*. martinfowler.com, 2005.

[9] John Hopcroft, Rajeev Motwani, and Jeffrey Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson, 2006.

[10] Ernst Ising. *Contribution to the Theory of Ferromagnetism*. Zeitschrift für Physik, 31:253–258, 1925.

[11] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[12] Leslie Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, 21(7):558–565, 1978.

[13] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 2008.

[14] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[15] Marc Mézard, Giorgio Parisi, and Miguel Virasoro. *Spin Glass Theory and Beyond*. World Scientific, 1987.

[16] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[17] Mogens Nielsen, Gordon Plotkin, and Glynn Winskel. *Petri Nets, Event Structures and Domains*. Theoretical Computer Science, 13(1):85–108, 1995.

[18] Ángel Rivas, Susana F. Huelga, and Martin B. Plenio. *Entanglement and Non-Markovianity of Quantum Evolutions*. Physical Review Letters, 105(5):050403, 2010.

[19] Federico Settimo, Andrea Smirne, Kimmo Luoma, Bassano Vacchini, Jyrki Piilo, and Dariusz Chruściński. *Divisibility of Dynamical Maps: Schrödinger Versus Heisenberg Picture*. PRX Quantum, 7(1), 2026. https://doi.org/10.1103/6dt2-sq44

[20] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. *Conflict-Free Replicated Data Types*. In *Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2011.

[21] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2013.

[22] Glynn Winskel. *Event Structures*. In *Advances in Petri Nets 1986*, Lecture Notes in Computer Science 255:325–392. Springer, 1987.

[23] Glynn Winskel and Mogens Nielsen. *Models for Concurrency*. In *Handbook of Logic in Computer Science*. Oxford University Press, 1993.